

ui.footer 全面详细解析

`ui.footer` 是 NiceGUI 框架中用于构建页面底部区域的核心布局组件，基于 Quasar 框架的 Footer 组件封装，主要用于展示版权信息、导航链接、联系方式等辅助内容。该组件默认固定在页面底部，支持视觉定制、状态控制和功能扩展，继承自 `valueElement`、`Element`、`Visibility` 基类，具备通用的元素操作能力，是构建完整页面布局的重要组成部分。以下从核心特性、初始化参数、属性与方法、使用示例、注意事项等方面进行全面解析。

一、核心特性

- 布局定位**：默认固定在页面底部（`fixed=True`），滚动页面时保持可见，也可配置为随内容滚动。
- 视觉定制**：支持显示顶部边框（`bordered=True`），可通过 `style`、`classes` 灵活调整背景色、内边距、文字样式等。
- 状态可控**：支持默认展开 / 折叠（`value` 参数）、动态显示 / 隐藏、状态切换，适配不同页面场景需求。
- 功能扩展**：可与链接、按钮、文本等组件深度集成，支持事件监听、数据绑定、动态资源添加等高级功能。
- 轻量灵活**：初始化参数简洁，核心功能聚焦底部区域展示，无冗余配置，易于快速上手。

二、初始化参数 (Initializer)

`ui.footer` 的初始化参数用于配置组件基础行为与样式，所有参数均为可选，默认值已适配通用场景，详细说明如下：

参数名	类型	说明	默认值
value	bool	初始展开状态：- <code>True</code> ：页面加载时默认展开（footer 通常为常驻展示，此参数控制初始显示）- <code>False</code> ：页面加载时默认折叠	True
fixed	bool	定位方式控制：- <code>True</code> ：固定在页面底部，滚动页面时位置不变- <code>False</code> ：随页面内容一起滚动，滚动到页面底部后可见	True
bordered	bool	是否显示顶部边框，用于区分 footer 与页面主体内容，边框样式可通过 <code>style</code> 补充定制	False

三、核心属性 (Properties)

继承自基类的核心属性，支持动态获取和修改组件状态与配置，核心属性如下：

属性名	类型	说明
classes	Classes[Self]	组件的 HTML 类名，支持通过 Tailwind/Quasar 类调整布局（如 <code>flex justify-between</code> ）和样式（如 <code>p-4</code> ），支持链式调用修改
client	Client	组件所属的客户端实例，关联 websocket 连接、布局上下文等，用于多客户端联动等场景

属性名	类型	说明
html_id	str	组件在 HTML DOM 中的唯一 ID (2.16.0 版本新增)，可用于直接通过 DOM 操作组件
is_deleted	bool	组件是否已被删除（只读属性），用于判断组件生命周期状态
is_ignoring_events	bool	组件是否正在忽略事件（只读属性），用于事件控制逻辑
parent_slot	Slot None	组件的父插槽（可设置），用于调整组件在父容器中的插槽位置，适配复杂布局
props	Props[Self]	组件的 Quasar 原生属性，支持链式调用添加 / 删除（如 <code>props('rounded')</code> ），扩展组件原生能力
style	Style[Self]	组件的内联 CSS 样式，支持链式调用（如 <code>style('background-color: #333').style('color: #fff')</code> ），精准控制外观
value	BindableProperty	组件当前展开 / 折叠状态（ <code>True</code> 为展开， <code>False</code> 为折叠），可绑定数据动态控制
visible	BindableProperty	组件的可见性（ <code>True</code> 为显示， <code>False</code> 为隐藏），可绑定数据动态控制

四、关键方法 (Methods)

`ui.footer` 继承了基类的丰富方法，支持组件操作、状态管理、事件绑定等，按功能分类解析核心方法：

1. 基础状态操作

方法名	参数	说明
<code>show()</code>	-	显示 footer 组件（仅控制可见性，不改变展开 / 折叠状态）
<code>hide()</code>	-	隐藏 footer 组件
<code>toggle()</code>	-	切换 footer 的展开 / 折叠状态（展开 ↔ 折叠）
<code>delete()</code>	-	删除 footer 组件及所有子元素，释放资源
<code>clear()</code>	-	清除 footer 内的所有子元素（保留 footer 本身，可重新添加内容）
<code>update()</code>	-	同步组件状态到客户端，修改样式、属性或子元素后需调用，确保变更生效
<code>set_value()</code>	<code>value: Any</code>	手动设置 footer 的展开 / 折叠状态（如 <code>set_value(True)</code> 强制展开）
<code>set_visibility()</code>	<code>visible: bool</code>	手动设置 footer 的可见性（如 <code>set_visibility(False)</code> 强制隐藏）

2. 样式与属性修改

方法名	核心参数	说明
default_classes()	add/remove/toggle/replace	批量修改同类组件的默认类名（需在实例化前调用，作用于所有 <code>ui.footer</code> ）
default_props()	add/remove	批量修改同类组件的默认 Quasar 属性（需在实例化前调用）
default_style()	add/remove/replace	批量修改同类组件的默认 CSS 样式（需在实例化前调用，如全局设置背景色）
style()	css_str	设置内联样式，优先级高于 <code>classes</code> ，支持多轮链式调用
classes()	class_str	设置 HTML 类名，可结合 Tailwind 实现快速布局（如 <code>classes('flex items-center justify-between p-4')</code> ）
props()	props_str	设置 Quasar 原生属性，扩展组件功能（如 <code>props('bordered rounded')</code> 添加边框和圆角）

3. 数据绑定方法

支持将 footer 状态与 Python 对象属性动态绑定，实现单向 / 双向同步，简化状态管理：

方法名	核心参数	说明
bind_value()	target_object, target_name	双向绑定：footer 展开状态与目标对象属性同步（一方修改，另一方自动更新）
bind_value_from()	target_object, target_name	单向绑定（从目标到组件）：目标对象属性变化时，同步更新 footer 展开状态
bind_value_to()	target_object, target_name	单向绑定（从组件到目标）：footer 展开状态变化时，同步更新目标对象属性
bind_visibility()	target_object, target_name	双向绑定：footer 可见性与目标对象属性同步
bind_visibility_from()	target_object, target_name	单向绑定（从目标到组件）：目标对象属性变化时，同步更新 footer 可见性
bind_visibility_to()	target_object, target_name	单向绑定（从组件到目标）：footer 可见性变化时，同步更新目标对象属性

4. 事件绑定方法

方法名	核心参数	说明
on()	type, handler, js_handler	绑定组件事件（如 <code>click</code> 、 <code>update:model-value</code> 等），支持 Python 处理器（服务端处理）或 JavaScript 处理器（客户端处理），2.18.0+ 版本可同时指定两者
on_value_change()	callback	绑定展开状态变化事件：当 footer 从展开变为折叠（或反之）时触发回调函数，回调参数包含事件详情（如 <code>e.value</code> 为新状态）

5. 其他实用方法

方法名	核心参数	说明
tooltip()	text	为 footer 添加悬浮提示文本（鼠标悬浮时显示）
mark()	*markers	为组件添加标记（如 <code>mark('page-footer copyright')</code> ），用于测试查询或依赖管理
move()	target_container, target_index	将 footer 移动到其他父容器中，调整布局结构
add_resource()	path	为 footer 添加静态资源（如 CSS/JS 文件），扩展样式或功能
add_dynamic_resource()	name, function	为 footer 添加动态资源（通过函数返回资源响应），支持动态生成内容
get_computed_prop()	prop_name, timeout	异步获取组件的计算属性（需 <code>await</code> 调用），如获取 footer 实际高度
run_method()	name, *args	在客户端执行组件原生方法（如 Quasar Footer 的 <code>show()</code> 方法），需 <code>await</code> 调用

五、使用示例

1. 基础用法：简单版权 footer

```
from nicegui import ui

@ui.page('/basic_footer')
def basic_footer_demo():
    # 页面主体内容
    [ui.label(f'页面内容行 {i}') for i in range(50)]

    # 创建底部 footer：固定定位、带边框、深色背景
    with ui.footer(
        fixed=True,
        bordered=True,
        style='background-color: #2c3e50; color: #ecf0f1; padding: 1rem'
```

```
) .classes('flex justify-center items-center'):
    ui.label('© 2025 我的应用 版权所有')

ui.run()
```

2. 高级用法：多区域布局与状态绑定

```
from nicegui import ui

class AppState:
    def __init__(self):
        self.footer_visible = True # 控制 footer 可见性

@ui.page('/advanced_footer')
def advanced_footer_demo():
    state = AppState()

    # 创建 footer 并绑定可见性
    footer = ui.footer(
        fixed=True,
        bordered=True,
        style='background-color: #f8f9fa'
    ).bind_visibility(state, 'footer_visible')

    with footer.classes('flex justify-between items-center p-4'):
        # 左侧：版权信息
        ui.label('© 2025 我的应用')
        # 中间：导航链接
        with ui.row().classes('mx-4'):
            ui.link('关于我们', '/about').classes('text-blue-600 hover:underline mx-2')
            ui.link('隐私政策', '/privacy').classes('text-blue-600 hover:underline mx-2')
            ui.link('联系我们', '/contact').classes('text-blue-600 hover:underline mx-2')
        # 右侧：控制按钮
        ui.checkbox('显示底部栏', value=state.footer_visible).bind_value(state,
            'footer_visible')

    # 页面主体内容
    with ui.column().classes('p-6'):
        ui.label('页面主体').classes('text-2xl font-bold mb-4')
        ui.button('隐藏 footer', on_click=lambda: footer.hide()).props('flat')
        ui.button('显示 footer', on_click=lambda: footer.show()).props('flat')
        [ui.label(f'主体内容行 {i}') for i in range(50)]

ui.run()
```

3. 实用场景：响应式 footer (含联系方式与回到顶部)

```
from nicegui import ui

@ui.page('/responsive_footer')
def responsive_footer_demo():
```

```
# 创建 footer: 响应式布局, 小屏幕自动换行
with ui.footer(
    fixed=True,
    bordered=True,
    style='background-color: #34495e; color: #fff'
).classes('flex flex-col md:flex-row justify-between items-center p-4'):
    # 左侧: 联系方式 (小屏幕居上, 大屏居左)
    with ui.column().classes('md:flex-row md:items-center mb-4 md:mb-0'):
        ui.label('联系我们: ').classes('mr-2')
        ui.label('service@example.com').classes('mr-4')
        ui.label('123-4567-8910')

    # 右侧: 回到顶部按钮 (小屏幕居下, 大屏居右)
    def scroll_to_top():
        ui.run_javascript('window.scrollTo({top: 0, behavior: "smooth"})')

    ui.button('回到顶部', on_click=scroll_to_top).props('flat color=white')

# 页面主体内容 (长文本用于测试滚动)
with ui.column().classes('p-6'):
    ui.label('响应式 footer 演示').classes('text-2xl font-bold mb-4')
    [ui.label(f'主体内容行 {i}') for i in range(100)]

ui.run()
```

六、注意事项

- 固定定位与页面底部内边距:** `fixed=True` 时, footer 会覆盖页面底部内容, 建议为页面主体添加底部内边距 (如 `classes('pb-16')`, 对应 footer 高度 4rem), 确保内容不被遮挡。
- 响应式布局适配:** 小屏幕下建议使用 `flex-col` 类让 footer 内容垂直排列, 避免内容溢出; 可结合 Tailwind 的响应式类 (如 `md:flex-row`) 实现大屏横向布局、小屏纵向布局。
- 样式优先级:** `style` 方法设置的内联样式优先级高于 `classes` 中的类样式, 若需覆盖内联样式, 可在类中使用 `!important` 关键字 (如 `classes('background-color: #fff !important')`)。
- 状态绑定的严格模式:** 绑定数据时, `strict` 参数 (默认 `None`) 会检查目标对象是否存在指定属性, 非字典对象建议开启 `strict=True` 以避免错误。
- 版本兼容性:** `html_id` 属性需 NiceGUI 2.16.0+ 版本, `js_handler` 支持同时指定 Python 与 JS 处理器需 2.18.0+ 版本, 使用时需确认版本匹配。
- 边框样式定制:** `bordered=True` 仅显示顶部边框, 若需修改边框颜色、宽度, 可通过 `style('border-top: 2px solid #ddd')` 补充配置。

七、扩展参考

- 更多 Quasar 原生属性可通过 `props()` 方法传递, 详细支持的属性可参考 [Quasar Footer 文档](#)。
- 样式定制可结合 Tailwind CSS 类 (如 `flex`、`justify-between`、`p-4`) 实现快速布局, 或通过 `style` 方法设置内联样式 (如 `background-color`、`color`、`padding`)。
- 若需实现 footer 滚动显示 / 隐藏 (滚动时隐藏、停止滚动时显示), 可结合 `ui.on_scroll` 事件监听页面滚动状态, 动态调用 `show()` / `hide()` 方法。

ui.footer

在 NiceGUI 中，`ui.footer()` 是专门用于创建**页面页脚**的组件，它封装了页脚的基础样式和布局特性，能够快速实现固定在页面底部、自适应宽度、内容居中的页脚效果，同时支持自定义样式、响应式设计和内容嵌套。本文将从**基础使用**、**核心特性**、**样式定制**、**响应式适配**和**实战示例**五个维度，详细阐述 `ui.footer()` 的使用方法。

一、`ui.footer()` 的基础使用

`ui.footer()` 是一个容器组件，本质上基于 HTML 的 `<footer>` 标签实现，默认具有**宽度占满页面**、**内容垂直居中**、**底部定位**的基础特性。使用时通过**上下文管理器**（`with` 语句）向其中添加子组件（如文本、链接、按钮等），是创建页脚的最简方式。

1. 最简示例

```
from nicegui import ui

# 页面主内容
ui.label('这是页面主内容').classes('text-2xl p-10')

# 基础页脚
with ui.footer():
    ui.label('© 2025 NiceGUI 示例 - 保留所有权利')

ui.run()
```

效果：页脚固定在页面底部（若主内容高度不足屏幕高度，页脚贴紧屏幕底部；若主内容超出屏幕高度，页脚跟随内容滚动到页面最下方），内容默认水平居中，占满页面宽度。

2. 嵌套多元素

页脚支持嵌套文本、链接、按钮、图标等任意 NiceGUI 组件，通过 `ui.row()` / `ui.column()` 实现内容的水平 / 垂直排列：

```
from nicegui import ui

# 主内容
ui.column().style('min-height: 80vh;').label('页面主内容') # 模拟主内容高度

# 带多元素的页脚
with ui.footer():
    # 水平排列内容
    with ui.row().classes('w-full justify-between items-center px-6'):
        # 左侧文本
        ui.label('© 2025 我的网站')
        # 右侧链接组
        with ui.row().classes('gap-4'):
            ui.link('关于我们', '/about')
            ui.link('隐私政策', '/privacy')
            ui.link('联系我们', '/contact')
        # 右侧按钮
        ui.button('反馈', icon='feedback')
```

```
ui.run()
```

关键配置：

- `w-full`：行容器宽度占满页脚；
- `justify-between`：内容左右两端对齐；
- `items-center`：子组件垂直居中；
- `px-6`：左右内边距，避免内容贴边。

二、`ui.footer()` 的核心特性

`ui.footer()` 作为专用的页脚组件，具有以下核心特性，区别于普通容器（如 `ui.element('footer')`）：

1. **默认样式**：自带 `bg-gray-100`（浅灰色背景）、`py-4`（上下内边距）、`text-sm`（小号文本）的 Tailwind 类，无需手动设置基础样式；
2. **语义化标签**：基于 HTML5 的 `<footer>` 标签，符合网页语义化规范，利于 SEO；
3. **容器特性**：继承 NiceGUI 容器的所有能力，支持添加子组件、设置样式、绑定事件；
4. **自适应宽度**：默认宽度为 100%，跟随页面宽度自适应调整。

三、`ui.footer()` 的样式定制

`ui.footer()` 的样式可通过 `classes()`（Tailwind 类）和 `style()`（自定义 CSS）两种方式定制，覆盖背景色、内边距、文字样式、高度等所有视觉属性。

1. 基础样式定制

修改页脚的背景色、文字颜色、内边距和高度：

```
from nicegui import ui

with ui.footer() \
    .classes('bg-gray-900 text-white py-6') # 深色背景、白色文字、加大上下内边距
    .style('height: 100px;') # 自定义高度
    ui.label('深色页脚示例 - 自定义样式').classes('text-lg')

ui.run()
```

2. 内容对齐方式

通过 Tailwind 的 Flex 布局类（`justify-center`/`justify-start`/`justify-end`/`justify-between`）控制页脚内容的对齐：

```
from nicegui import ui

with ui.footer().classes('bg-blue-50 py-4'):
    # 内容左对齐
    with ui.row().classes('w-full justify-start px-4'):
        ui.label('左对齐页脚')
```

```
# 分隔线
ui.separator()

with ui.footer().classes('bg-green-50 py-4 mt-10'):
    # 内容两端对齐
    with ui.row().classes('w-full justify-between px-4'):
        ui.label('左侧文本')
        ui.label('右侧文本')

ui.run()
```

3. 添加边框和阴影

为页脚添加顶部边框、阴影，增强视觉层次感：

```
from nicegui import ui

with ui.footer().classes('bg-white py-4 border-t-2 border-gray-200 shadow-md'):
    # border-t-2: 顶部2px边框; shadow-md: 中等阴影
    ui.label('带边框和阴影的页脚').classes('text-gray-700')

ui.run()
```

四、`ui.footer()` 的响应式适配

页脚的内容往往需要根据屏幕尺寸（手机、平板、桌面）调整布局，`ui.footer()` 结合 Tailwind 的**响应式前缀**（`sm:` / `md:` / `lg:`）可快速实现适配。

1. 移动端内容堆叠

桌面端水平排列的内容，在移动端垂直堆叠，避免内容挤在一起：

```
from nicegui import ui

with ui.footer().classes('bg-gray-800 text-white py-4'):
    # 桌面端：水平两端对齐；移动端：垂直列布局
    with ui.row().classes('w-full px-4 md:flex md:justify-between sm:block'):
        # 左侧文本
        ui.label('@ 2025 响应式页脚').classes('mb-2 md:mb-0') # 移动端底部边距，桌面端取消
        # 右侧链接组
        with ui.row().classes('gap-4 md:justify-end sm:block sm:gap-2'):
            ui.link('关于', '/about').classes('block')
            ui.link('隐私', '/privacy').classes('block')
            ui.link('联系', '/contact').classes('block')

ui.run()
```

关键响应式类：

- `md:flex / sm:block`：桌面端用 Flex 布局，移动端用块级布局；

- `md:justify-between`：桌面端内容两端对齐；
- `mb-2 md:mb-0`：移动端添加底部边距，桌面端取消。

2. 移动端隐藏部分内容

对于非核心内容（如桌面端的“技术支持”文本），在移动端隐藏以简化页脚：

```
from nicegui import ui

with ui.footer().classes('bg-gray-100 py-4'):
    with ui.row().classes('w-full justify-between px-4'):
        ui.label('© 2025 我的网站')
        # 仅桌面端显示
        ui.label('技术支持: NiceGUI').classes('hidden lg:block')
        ui.link('联系我们', '/contact')

ui.run()
```

关键类：`hidden lg:block`（默认隐藏，大屏幕（≥1024px）时显示）。

五、`ui.footer()` 的高级用法

1. 固定在页面底部（粘性页脚）

默认情况下，`ui.footer()` 会跟随页面内容滚动，若需要无论主内容高度如何，页脚始终固定在屏幕底部（粘性页脚），可通过 `position: fixed` 实现：

```
from nicegui import ui

# 主内容（高度不足屏幕）
ui.label('粘性页脚示例').classes('text-2xl p-10')

# 固定底部的页脚
with ui.footer() \
    .classes('bg-gray-900 text-white py-3') \
    .style('position: fixed; bottom: 0; left: 0; width: 100%; z-index: 10;'):
    # z-index: 确保页脚不被其他组件遮挡
    ui.label('固定在屏幕底部的页脚').classes('text-center')

# 主内容底部边距：避免内容被固定页脚遮挡
ui.add_body_html('<style>body { padding-bottom: 60px; }</style>')

ui.run()
```

注意：需为页面主体（`body`）添加底部内边距（`padding-bottom`），否则页面最下方的内容会被固定页脚遮挡。

2. 带图标的页脚

结合 NiceGUI 的图标组件 (`ui.icon()`)，为页脚添加社交图标、功能图标，增强视觉效果：

```
from nicegui import ui

with ui.footer().classes('bg-gray-100 py-4'):
    with ui.row().classes('w-full justify-center items-center gap-6 px-4'):
        # 文本
        ui.label('关注我们: ')
        # 社交图标 (带链接)
        ui.link('', 'https://github.com', new_tab=True).add(ui.icon('github').classes('text-2xl text-gray-700'))
        ui.link('', 'https://twitter.com',
new_tab=True).add(ui.icon('twitter').classes('text-2xl text-blue-500'))
        ui.link('', 'https://facebook.com',
new_tab=True).add(ui.icon('facebook').classes('text-2xl text-blue-800'))
        # 分隔线
        ui.separator().classes('h-8')
        # 版权信息
        ui.label('@ 2025 我的网站').classes('text-sm text-gray-500')

ui.run()
```

3. 多列布局的页脚

对于内容丰富的页脚（如电商网站的页脚），可通过 `ui.grid()` 或 `ui.column()` 实现多列布局，分类展示内容：

```
from nicegui import ui

with ui.footer().classes('bg-gray-800 text-white py-8'):
    # 4列网格布局，移动端堆叠为1列
    with ui.grid(columns=4, gap=4).classes('w-full max-w-6xl mx-auto px-4 sm:grid-cols-2 lg:grid-cols-4'):
        # 第一列：关于我们
        with ui.column().classes('gap-2'):
            ui.label('关于我们').classes('text-lg font-bold mb-2')
            ui.link('公司简介', '/about')
            ui.link('团队介绍', '/team')
            ui.link('发展历程', '/history')
        # 第二列：服务支持
        with ui.column().classes('gap-2'):
            ui.label('服务支持').classes('text-lg font-bold mb-2')
            ui.link('帮助中心', '/help')
            ui.link('售后服务', '/service')
            ui.link('常见问题', '/faq')
        # 第三列：联系方式
        with ui.column().classes('gap-2'):
            ui.label('联系方式').classes('text-lg font-bold mb-2')
            ui.label('电话: 123-4567-8901')
            ui.label('邮箱: contact@example.com')
            ui.label('地址: 北京市海淀区')
```

```
# 第四列：订阅
with ui.column().classes('gap-2'):
    ui.label('订阅推送').classes('text-lg font-bold mb-2')
    ui.input(placeholder='输入您的邮箱').classes('w-full')
    ui.button('订阅', color='blue').classes('w-full mt-2')

# 底部版权栏
with ui.row().classes('w-full mt-6 border-t border-gray-700 pt-4 text-center justify-center'):
    ui.label('© 2025 我的电商网站 - 保留所有权利')

ui.run()
```

关键配置:

- `max-w-6xl mx-auto`: 网格容器最大宽度为 6xl，水平居中，避免页脚内容在超大屏幕上过度拉伸；
- `sm:grid-cols-2 lg:grid-cols-4`: 移动端 2 列，桌面端 4 列的响应式网格。

六、 `ui.footer()` 与普通容器的区别

很多开发者会用 `ui.element('footer')` 或 `ui.row()` 替代 `ui.footer()`，二者的核心区别如下：

特性	<code>ui.footer()</code>	普通容器 (<code>ui.element('footer')</code>)
基础样式	自带浅灰背景、内边距等样式	无默认样式，需手动设置
语义化	基于 HTML5 <code><footer></code> 标签	需手动指定标签，灵活性高但语义化弱
使用便捷性	开箱即用，适合快速开发	需手动配置样式，适合高度定制
容器能力	继承所有容器特性	与 <code>ui.footer()</code> 完全一致

总结：若只需快速实现基础页脚，优先使用 `ui.footer()`；若需要高度定制的页脚（如无默认样式、特殊布局），可使用 `ui.element('footer')` 手动构建。

七、 总结

`ui.footer()` 是 NiceGUI 中实现页脚的**专用、高效组件**，通过默认样式减少了基础配置工作，同时支持灵活的样式定制和内容嵌套。其核心使用思路为：

1. **基础使用：**通过 `with` 语句向页脚中添加文本、链接、图标等子组件；
2. **样式定制：**利用 Tailwind 类和自定义 CSS 修改背景、对齐、内边距等；
3. **响应式适配：**结合 Tailwind 响应式前缀实现移动端 / 桌面端的布局调整；
4. **高级需求：**通过 `position: fixed` 实现粘性页脚，通过网格 / 列布局实现多列内容展示。

无论是简单的版权信息页脚，还是复杂的多列电商页脚，`ui.footer()` 都能以简洁的 Python 代码实现，是 NiceGUI 页面布局中不可或缺的组件。